

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Scenario-Based

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1513

COURSE OUTCOME COVERED

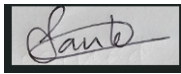
CO2: *Analyse virtualisation architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2)
Question Count: 5

Total Marks: 25

Code of Conduct

I, SANDEEP K / 192521253 (Name / Reg No), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.



I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the student:

Assessment Tasks Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering.
Design a cloud architecture using scalability and caching techniques.
Explain how load balancing and auto-scaling can ensure smooth streaming.
Discuss cost optimization strategies for such high-demand workloads.
2. A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments.
Propose a solution using container orchestration and multi-cloud strategies.
Explain how service discovery and scaling can be handled efficiently.
Highlight the benefits and risks of a multi-cloud deployment model.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud. The organization needs a robust backup and disaster recovery strategy. Design a solution using cloud-native backup, replication, and versioning.

Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved. Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data. Some applications must remain on-premises due to compliance requirements.

Design a hybrid architecture that ensures secure communication between environments.

Explain how identity management and encryption should be implemented.

Discuss challenges in managing hybrid cloud environments.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application. The system must continue operating even if one region goes down. Propose a multi-region deployment strategy in the cloud. Explain how data replication and failover mechanisms should be configured.

Discuss monitoring and alerting strategies for proactive issue detection.

Scenario-Based Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and wellorganized response	Generally clear	Somewhat unclear	Poorly presented

SIMATS ENGINEERING

ASSESSMENT TOOL 2 : Scenario-Based

Course: Cloud Computing and Big Data Analytics (CSA15) | CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments (BL3)

Scenario 1: Video Streaming Platform Migration

Understanding of the Scenario

A media company wants to move its video streaming service to the cloud so it can serve millions of concurrent viewers with minimal buffering. The core challenges are elastic capacity, low-latency content delivery, and cost control under highly variable, spiky demand (e.g., new releases, live events).

Cloud Architecture Design

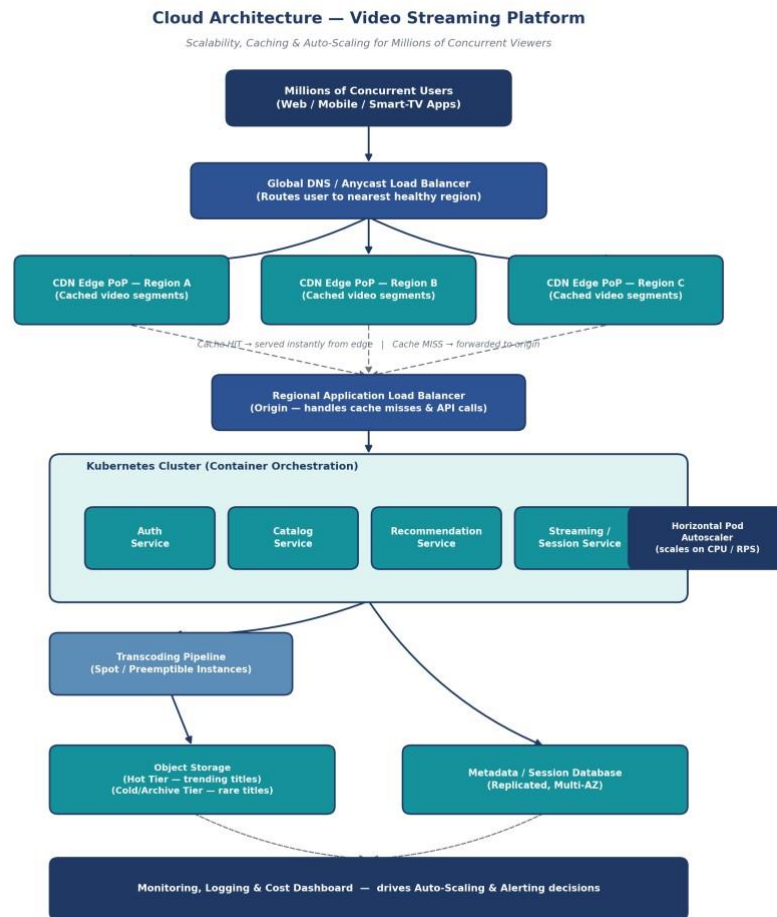


Figure 1: End-to-end cloud architecture for the video streaming platform

Content delivery layer: A CDN with edge Points of Presence (PoPs) caches video segments close to users. A cache hit is served instantly from the edge; a cache miss is forwarded to the origin, cutting latency and origin load.

Compute layer: A regional application load balancer fronts a Kubernetes cluster running containerized microservices — Auth, Catalog, Recommendation, and Streaming/Session — with a Horizontal Pod Autoscaler scaling pods based on CPU and request rate.

Transcoding layer: A separate transcoding pipeline runs on spot/preemptible instances, decoupled from the live-serving path so cost-driven interruptions never affect viewers.

Storage layer: Master video files live in object storage with lifecycle tiering — hot tier for trending content, cold/archive tier for rarely watched titles — alongside a replicated metadata/session database.

Streaming protocol: Adaptive bitrate streaming (HLS/DASH) lets the client player switch quality based on available bandwidth, directly reducing buffering.

Observability: A monitoring, logging, and cost dashboard feeds the auto-scaler and alerting pipeline, closing the loop between demand and capacity.

Load Balancing and Auto-Scaling

- A global (DNS/Anycast-based) load balancer routes users to the nearest healthy region; within a region, an application load balancer distributes requests across service instances.
- Auto-scaling groups/Horizontal Pod Autoscalers monitor CPU, request rate, and queue depth, and add or remove instances automatically — scaling out before a live event and scaling in afterward to save cost.
- Health checks remove unhealthy nodes from rotation automatically, preventing buffering caused by overloaded or failed instances.
- Together, load balancing spreads traffic evenly while auto-scaling ensures capacity always matches demand, keeping response times and buffering low even during traffic spikes.

Cost Optimization Strategies

- Use a mix of reserved/committed instances for predictable baseline load and spot/preemptible instances for burst transcoding jobs.
- Cache aggressively at the CDN edge to reduce origin egress and compute costs.
- Apply storage tiering/lifecycle policies to move cold content to cheaper archival storage.
- Right-size instances continuously using auto-scaling and monitoring dashboards to avoid overprovisioning.
- Use pay-as-you-go transcoding services only when needed instead of always-on dedicated transcoding clusters.

Decision Justification

This design directly targets the stated requirements: CDN plus adaptive bitrate streaming minimizes buffering; container orchestration with auto-scaling and load balancing gives elastic capacity for millions of concurrent users; and tiered storage with mixed pricing models controls cost without sacrificing availability.

Scenario 2: Multi-Cloud Container Deployment for a Fintech Startup (5 Marks)

Understanding of the Scenario

A fintech startup runs containerized applications across multiple cloud providers but struggles to keep deployments consistent and manageable. The issue is a lack of a unifying orchestration and governance layer across heterogeneous cloud environments.

Proposed Solution

Container orchestration: Adopt Kubernetes as the common orchestration layer across all providers, since it is cloud-agnostic and supported by every major provider (EKS, AKS, GKE) as well as selfmanaged clusters.

Multi-cloud management: Use a fleet/federation management tool (e.g., a multi-cluster management plane) to apply the same deployment manifests, policies, and configurations consistently to every cluster, avoiding provider-specific drift.

CI/CD pipeline: Use GitOps (e.g., a Git repository as the single source of truth with automated sync agents) so every cluster pulls the same versioned configuration, ensuring consistency.

Infrastructure as Code: Provision infrastructure with Terraform or similar tools so environments are reproducible and auditable across providers.

Service Discovery and Scaling

- Use a service mesh (e.g., Istio/Linkerd) to provide uniform service discovery, traffic routing, and mutual TLS across clusters regardless of which cloud they run on.
- Each cluster runs its own Horizontal Pod Autoscaler for local scaling; a global traffic manager shifts load between clouds based on latency, cost, or capacity.
- A shared service registry lets services on one cloud discover and call services on another securely.

Benefits and Risks of Multi-Cloud

Benefits:

- Avoids vendor lock-in and gives negotiating leverage on pricing.
- Improves resilience — an outage at one provider doesn't take down the whole platform.
- Allows choosing the best-fit service from each provider (e.g., specific ML or database offerings).

Risks:

- Increased operational complexity and need for cross-cloud expertise.
- Data transfer (egress) costs between clouds can be significant.
- Harder to maintain consistent security posture and compliance across providers, which is critical for fintech regulatory requirements.

Decision Justification

Kubernetes plus GitOps and a service mesh directly solves the stated consistency problem by making deployment and networking behavior identical regardless of the underlying cloud, while still preserving the flexibility and resilience that motivated the multi-cloud choice in the first place.

Scenario 3: Backup and Disaster Recovery for an E-Learning Platform (5 Marks)

Understanding of the Scenario

An e-learning platform suffered data loss from accidental deletion, showing it lacks a proper backup and disaster recovery (DR) strategy. The organization needs safeguards against accidental, malicious, and infrastructure-level data loss.

Proposed Solution

Backup strategy: Implement automated, scheduled backups (daily incremental, weekly full) of databases and file storage, stored in a separate storage account/region from production data.

Versioning: Enable object versioning on cloud storage so every prior version of a file is retained and accidental deletions/overwrites can be rolled back instantly.

Replication: Use cross-region (or cross-zone at minimum) replication so a copy of data always exists in a geographically separate location.

Soft-delete / recycle bin: Enable a soft-delete policy with a retention window (e.g., 30 days) so deleted objects can be recovered before permanent purge.

RTO and RPO

- Recovery Point Objective (RPO) — the maximum acceptable data loss, measured in time — is achieved by frequent, automated backups and near-real-time replication; e.g., continuous replication can reduce RPO to minutes.
- Recovery Time Objective (RTO) — the maximum acceptable downtime — is achieved through automated failover to a standby/replica environment and pre-tested recovery runbooks, minimizing manual recovery steps.
- Both targets should be defined per data criticality tier (e.g., student records vs. cached course thumbnails) so resources are focused where the impact of loss is highest.

Steps to Ensure Business Continuity

- Classify data by criticality and assign RTO/RPO targets to each class.
- Automate backups and test restores regularly (a backup is only useful if a restore has actually been verified).
- Maintain a documented, rehearsed DR runbook and conduct periodic failover drills.
- Use infrastructure as code so environments can be rebuilt quickly in a new region if needed.
- Monitor and alert on backup job failures so gaps are caught immediately, not during an actual incident.

Decision Justification

Versioning and soft-delete directly prevent the exact accidental-deletion scenario described, while crossregion replication and tested runbooks ensure the platform can meet defined RTO/RPO targets and keep operating through larger-scale failures, not just single-file mistakes.

Scenario 4: Hybrid Cloud for a Government Agency (5 Marks)

Understanding of the Scenario

A government agency must keep sensitive data on-premises for compliance while still benefiting from cloud scalability for non-sensitive workloads. The challenge is designing secure, well-governed connectivity between the two environments.

Hybrid Architecture Design

Workload placement: Keep regulated/sensitive workloads (citizen records, classified data) on-premises or in a private/government-community cloud; move public-facing, non-sensitive workloads (portals, informational sites, analytics on anonymized data) to the public cloud.

Secure connectivity: Connect the two environments using a dedicated private link (e.g., a direct connect/express route circuit or an IPsec VPN) rather than exposing traffic over the public internet.

Network segmentation: Use virtual private clouds/subnets with strict firewall and network ACL rules so only explicitly permitted traffic crosses between on-prem and cloud.

Identity Management and Encryption

- Federate identity using a single identity provider (e.g., SAML/OIDC-based SSO) so users and services authenticate once with consistent access policies across both environments.
- Apply role-based access control (RBAC) and the principle of least privilege uniformly on both sides.
- Encrypt data at rest using provider-managed or customer-managed keys, and encrypt data in transit with TLS across the private link.
- Use a centralized key management service so encryption keys for on-prem and cloud resources are governed under one policy.

Challenges in Managing Hybrid Cloud

- Maintaining consistent security and compliance policy across two different environments with different tools.
- Network latency and complexity of secure connectivity between on-prem and cloud.
- Data classification errors — misclassifying sensitive data as safe to move to the public cloud.
- Higher operational overhead from managing two operating models and skill sets simultaneously.
- Auditing and monitoring across both environments in a unified way for compliance reporting.

Decision Justification

Placing sensitive workloads on-premises while offloading non-sensitive workloads to the public cloud satisfies compliance without sacrificing scalability; federated identity, private connectivity, and centralized encryption ensure the two environments behave as one secure system rather than two loosely connected ones.

Scenario 5: High Availability and Fault Tolerance for a SaaS Application (5 Marks)

Understanding of the Scenario

A SaaS provider needs its application to remain available even if an entire cloud region fails. This requires a multi-region deployment with automated failover rather than reliance on a single region or availability zone.

Multi-Region Deployment Strategy

Active-active or active-passive regions: Deploy the application stack in at least two geographically separate regions. Active-active serves traffic from both regions simultaneously for maximum availability; active-passive keeps a standby region ready to take over, which is simpler but has a brief failover delay.

Global traffic routing: Use a global load balancer/DNS with health checks to route users to the nearest healthy region and automatically redirect traffic away from a failed region.

Stateless application tier: Design application servers to be stateless so any instance in any region can serve any request, simplifying failover.

Data Replication and Failover

- Use a database with built-in multi-region replication (synchronous for strong consistency where needed, asynchronous for lower latency elsewhere) so both regions have up-to-date data.
- Configure automated failover with health checks: if the primary region's database becomes unreachable, a replica is promoted automatically or via a fast, scripted runbook.

- Use quorum-based or consensus-based data stores for critical state to avoid split-brain issues during failover.

Monitoring and Alerting Strategy

- Implement synthetic monitoring/health checks that continuously probe each region from multiple external locations.
- Track key SLIs — latency, error rate, saturation — with alert thresholds tied to SLOs, so issues are caught before they become outages.
- Use distributed tracing and centralized logging aggregated across regions for fast root-cause analysis.
- Set up automated, escalating alerts (e.g., paging on-call engineers) and dashboards for real-time visibility across both regions.

Decision Justification

A multi-region active-active/active-passive design with replicated data and automated, health-checkdriven failover directly satisfies the requirement that the system keep operating if one region goes down, while proactive monitoring ensures issues are detected and escalated before they impact users.